

Agent Spec: Local AI Travel Intelligence Scraper

United Airlines + Delta + American + Partner Airlines + Price Aggregators

Resolved Configuration

Setting	Value
Language	Python 3.12+
OS	macOS Tahoe, Apple M4, 24 GB RAM
Browser automation	Playwright (Python)
Vision model	Ollama + LLaVA (fully local, no API key)
Authenticated scraping	Public pages only
Run mode	One-time run (scheduled re-crawl code present but commented out)
Output format	CSV with N/A for any field not found
Partner sites	Crawl directly
Price cross-reference	Google Flights + Kayak

Language Rationale

Python is the correct choice for this stack. United and all target sites are heavily JavaScript-rendered — Playwright Python handles this natively. The rest of the toolchain falls into Python naturally:

- playwright — browser automation + AX tree access
- ollama Python client — local vision inference via LLaVA
- pandas — CSV output + data cleaning
- beautifulsoup4 / lxml — HTML parsing layer
- sqlite3 (stdlib) — intermediate storage before CSV export

Node.js Playwright is equally capable at the browser layer, but the ML, data processing, and Ollama integration ecosystem is significantly weaker in JS. Python is the right call end-to-end.

Folder Structure

All files the script creates live in exactly two folders, both at the **same level as the script file**. This makes cleanup trivial — delete either folder and everything inside is gone.

```
your_project_folder/
|
├─ scraper.py                ← the main script
|
├─ screenshots/             ← ⚠ FILL IN PATH BELOW – temporary, deleted
after run                    after run
|   └─ (PNG files, auto-deleted on script completion)
|
├─ data/                    ← ⚠ FILL IN PATH BELOW – persistent output
    ├─ united_scraper.sqlite ← intermediate structured storage
    ├─ visit_log.jsonl       ← one JSON line per page visited
    ├─ failed_urls.log       ← URLs where both DOM and vision failed
    ├─ skipped_urls.log      ← URLs excluded by scope rules
    ├─ checkpoint.json       ← queue state (resume after crash)
    └─ output.csv            ← final clean output for algorithm
```

⚠ Path Configuration (fill these in before running)

In `scraper.py`, near the top of the file, there will be two clearly marked constants:

```
python

# =====
# CONFIGURE THESE PATHS BEFORE RUNNING
# Use absolute paths or paths relative to this script file.
# =====

SCREENSHOTS_DIR = "" # e.g. "/Users/yourname/projects/scraper/screenshots"
DATA_DIR        = "" # e.g. "/Users/yourname/projects/scraper/data"

# =====
```

Both directories will be created automatically if they don't exist. Screenshots are deleted at the end of a successful run. The data folder persists.

Prerequisites (what must be installed on the Mac before running)

The agent should generate a `setup.sh` and `requirements.txt` that handle all of this, but the human should be aware of these dependencies:

```
bash

# 1. Python 3.12+ (via Homebrew recommended on M4 Mac)
brew install python@3.12

# 2. Playwright browsers
pip install playwright
playwright install chromium # only Chromium needed

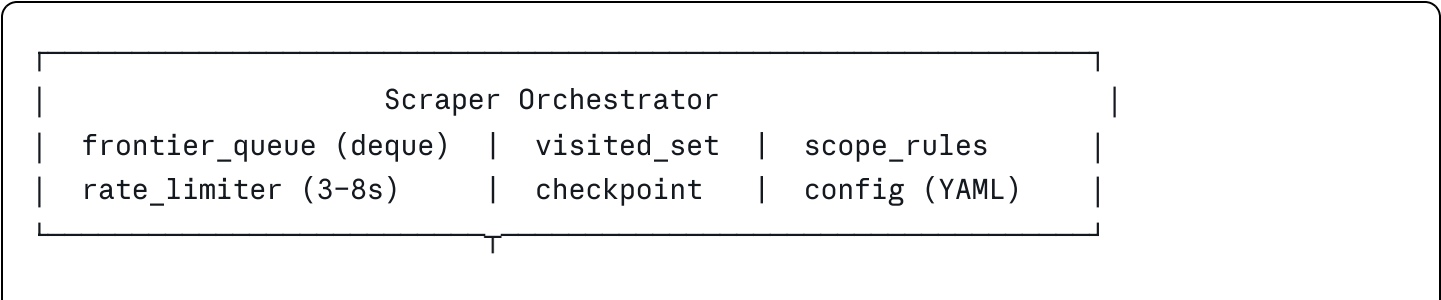
# 3. Python packages
pip install -r requirements.txt
# includes: playwright, beautifulsoup4, lxml, pandas, requests, ollama, pyyaml, apsc

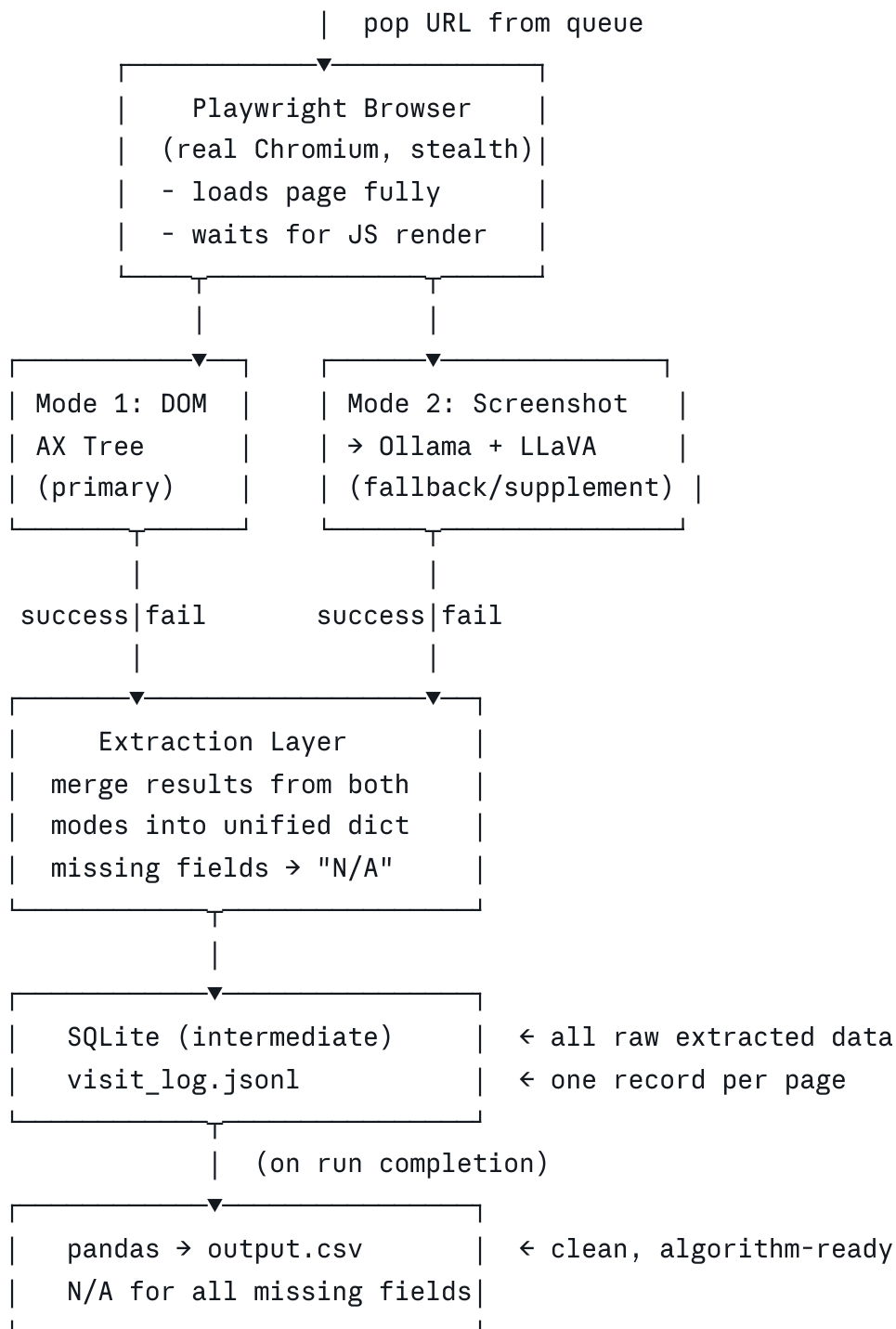
# 4. Ollama (local vision model runtime)
brew install ollama
ollama serve # start Ollama daemon (runs in background)
ollama pull llava # download LLaVA model (~4 GB, one-time)
# On M4 Mac, Ollama uses the Neural Engine – inference is fast and silent
```

M4 Mac notes:

- 24 GB unified memory is more than sufficient — LLaVA at 7B uses ~5-6 GB; Playwright + Python uses ~300-600 MB
- Ollama runs natively on Apple Silicon (arm64) — no Rosetta needed
- The Neural Engine handles vision inference efficiently; the fan should rarely spin
- Storage: expect ~5-10 GB for the LLaVA model + ~1-3 GB for data over a full crawl

Architecture Overview





Dual-Mode Extraction — Graceful Degradation Rules

This is the core reliability requirement. **Neither mode crashing should stop the script.**

Mode 1: DOM / Accessibility Tree

python

```

try:
    snapshot = await page.accessibility.snapshot()
    dom_data = parse_ax_tree(snapshot)
    dom_success = True
except Exception as e:
    dom_data = {}
    dom_success = False
    log.warning(f"[DOM_FAIL] {url} - {e}")

```

- Use `page.accessibility.snapshot()` to get the full AX tree
- Also query DOM directly for tables, definition lists, JSON-LD script tags, and `<meta>` tags
- If the page returns a Cloudflare challenge page, an empty tree, or a load timeout: log and continue

Mode 2: Screenshot + LLaVA (Ollama)

```

python

try:
    screenshot_path = SCREENSHOTS_DIR / f"page_{page_id}.png"
    await page.screenshot(path=screenshot_path, full_page=True)
    vision_data = query_llava(screenshot_path, context_prompt)
    vision_success = True
except Exception as e:
    vision_data = {}
    vision_success = False
    log.warning(f"[VISION_FAIL] {url} - {e}")
finally:
    # always attempt to delete the screenshot
    if screenshot_path.exists():
        screenshot_path.unlink()

```

Screenshot lifecycle:

- Screenshots are written to `SCREENSHOTS_DIR` during processing
- Each screenshot is deleted immediately after LLaVA has processed it (in the `finally` block)
- At the end of the full run, any remaining screenshots are swept and deleted
- If the run crashes mid-way, a cleanup function runs on next startup and deletes orphaned PNGs

LLaVA Prompt Strategy

LLaVA should be prompted with a structured context prompt that varies by page type. Example for an award pricing page:

You are extracting structured travel data from a screenshot of an airline website. Look for: award flight prices in miles, cash prices in USD, origin/destination airports, cabin class, partner airlines, transfer fees, and fuel surcharges. Return ONLY a JSON object with these keys: origin, destination, cabin, miles_cost, cash_cost_usd, taxes_fees_usd, partner_airline, is_partner_flight, notes. Use null for any field you cannot find.

Combined Result

```
python
page_record = {
    "url": url,
    "visited_at": datetime.utcnow().isoformat(),
    "dom_success": dom_success,
    "vision_success": vision_success,
    "dom_fail_reason": str(e) if not dom_success else None,
    "vision_fail_reason": str(e) if not vision_success else None,
    # merged data - vision fills gaps where DOM failed, DOM takes precedence
    "data": {**vision_data, **dom_data},
}
```

If **both** fail: URL is written to `failed_urls.log`. The script continues to the next URL without raising.

Sites to Crawl

Primary: United Airlines

Seed URL	Data target
<code>https://www.united.com</code>	Homepage, nav discovery

Seed URL	Data target
https://www.united.com/ual/en/us/fly/mileageplus.html	Program overview
https://www.united.com/ual/en/us/fly/mileageplus/awards/airline-partners.html	Partner award charts
https://www.united.com/ual/en/us/fly/mileageplus/earn/credit-cards.html	United credit cards
https://www.united.com/ual/en/us/fly/travel/airline-partners.html	Codeshare partners
https://www.united.com/ual/en/us/fly/mileageplus/awards/award-chart.html	Award chart

Scope: *.united.com | Max depth: 5 | Max pages: 800

Delta Air Lines

Seed URL	Data target
https://www.delta.com/us/en/skymiles/overview	SkyMiles program
https://www.delta.com/us/en/skymiles/airline-partners/overview	Partner airlines
https://www.delta.com/us/en/skymiles/how-to-earn-miles/credit-cards	Delta credit cards
https://www.delta.com/us/en/skymiles/redeem-miles/award-travel	Award redemption

Scope: *.delta.com | Max depth: 4 | Max pages: 400

American Airlines

Seed URL	Data target
https://www.aa.com/i18n/aadvantage-program/miles/redeem/award-travel/award-travel.jsp	AAdvantage awards

Seed URL	Data target
https://www.aa.com/i18n/aadvantage-program/partners/airline-partners.jsp	Partner airlines
https://www.aa.com/i18n/aadvantage-program/credit-cards/credit-cards.jsp	AA credit cards

Scope: [*.aa.com](#) | Max depth: 4 | Max pages: 400

Credit Card Transfer Partners

Site	Data target
https://creditcards.chase.com/travel-credit-cards/sapphire/reserve	UR transfer partners + ratios
https://www.americanexpress.com/en-us/rewards/membership-rewards/	MR transfer partners + ratios
https://www.citi.com/credit-cards/citi-thankyou-preferred-credit-card	Citi TY partners
https://www.capitalone.com/credit-cards/miles/	CapOne miles partners
https://www.biltrewards.com/points/transfer-partners	Bilt transfer partners

Scope: domain-locked per site | Max depth: 3 | Max pages: 100 each

Star Alliance Partner Airlines (crawl directly)

For each partner, scrape their award chart page and partner program page:

Airline	Award chart seed
ANA	https://www.ana.co.jp/en/us/anamileageclub/
Lufthansa	https://www.miles-and-more.com/us/en/earn/partners.html
Air Canada	https://www.aircanada.com/us/en/aco/home/aeroplan.html
Singapore Airlines	https://www.singaporeair.com/en_UK/us/ppsclub-krisflyer/
Turkish Airlines	https://www.turkishairlines.com/en-us/miles-and-smiles/
Swiss	https://www.swiss.com/us/en/fly-earn/miles-and-more

Scope: domain-locked | Max depth: 3 | Max pages: 80 each

Cash Price Aggregators

Site	Data target
https://www.google.com/travel/flights	Cash prices (use DOM; Google Flights is JS-heavy — vision fallback likely needed)
https://www.kayak.com/flights	Cash prices + fare class

Note on aggregators: Google Flights and Kayak are significantly more anti-bot than airline sites. Expect higher vision fallback rates. Use generous delays (8-15s) on these domains. If both modes fail consistently, log prominently and continue — cash price data from airline sites themselves is the secondary source.

Lounge Intelligence

Site	Data target
https://www.loungebuddy.com	Lounge locations, amenities, access methods
https://www.prioritypass.com/en/lounges	Priority Pass lounge list
https://www.united.com/ual/en/us/fly/travel/airport/united-clubs.html	United Club locations

Valuation Reference (read-only)

Site	Data target
https://thepointsguy.com/guide/monthly-valuations/	Cents-per-mile valuations per program
https://onemileatatime.com/guides/award-chart-guide/	Award chart analysis and tips

Data Schema — CSV Output

The final `output.csv` will have one row per data point with a `record_type` column. The algorithm should filter by `record_type` to get the relevant table. Use `N/A` (as a string) for any field that could not be extracted.

`record_type: award_flight`

Column	Description
<code>origin_iata</code>	3-letter airport code
<code>destination_iata</code>	3-letter airport code
<code>cabin</code>	economy / premium_economy / business / first
<code>miles_cost</code>	Miles required (integer or N/A)
<code>taxes_fees_usd</code>	Cash fees on top of miles
<code>fuel_surcharge_usd</code>	YQ surcharge if separately listed
<code>program</code>	united / delta / american / ana / etc.
<code>partner_operated</code>	true / false / N/A
<code>partner_airline</code>	Carrier operating the flight
<code>is_saver_award</code>	true / false / N/A (saver = cheapest tier)
<code>one_way_roundtrip</code>	one_way / roundtrip

Column	Description
<code>stopover_allowed</code>	true / false / N/A
<code>source_url</code>	Page this was extracted from
<code>extracted_at</code>	ISO 8601 timestamp
<code>dom_success</code>	true / false
<code>vision_success</code>	true / false

record_type: cash_flight

Column	Description
<code>origin_iata</code>	
<code>destination_iata</code>	
<code>cabin</code>	
<code>price_usd</code>	Total cash price
<code>fare_class</code>	Single letter fare bucket (Y, B, N, etc.)
<code>refundable</code>	true / false / N/A
<code>flight_number</code>	e.g. UA837
<code>airline</code>	Operating carrier
<code>departure_datetime</code>	ISO 8601
<code>arrival_datetime</code>	ISO 8601
<code>duration_minutes</code>	Integer
<code>stops</code>	0, 1, 2
<code>source</code>	google_flights / kayak / united / delta / american
<code>source_url</code>	
<code>extracted_at</code>	

record_type: transfer_partner

Column	Description
source_currency	e.g. Chase Ultimate Rewards
destination_program	e.g. United MileagePlus
transfer_ratio	e.g. 1:1 or 2:1.5
transfer_bonus_active	true / false
transfer_bonus_pct	e.g. 30 (for 30% bonus)
transfer_bonus_expiry	Date or N/A
min_transfer_units	Minimum units to transfer
transfer_time_hours	Typical hours to complete
source_url	
extracted_at	

record_type: credit_card

Column	Description
card_name	
issuer	Chase / Amex / Citi / CapOne / Bilt
points_currency	e.g. United MileagePlus / Chase UR
annual_fee_usd	
signup_bonus_points	
signup_spend_usd	Spend required for signup bonus
signup_window_days	
earn_rate_base	Miles/points per \$1 on general spend
earn_rate_travel	Miles/points per \$1 on travel

Column	Description
earn_rate_dining	Miles/points per \$1 on dining
lounge_access	true / false / N/A
lounge_network	Priority Pass / Amex Centurion / United Club / N/A
free_checked_bag	true / false
priority_boarding	true / false
source_url	
extracted_at	

record_type: lounge

Column	Description
airport_iata	
terminal	
lounge_name	
operated_by	
network	Priority Pass / Amex / United Club / Delta Sky Club / Admirals Club
access_via_cards	Comma-separated list of cards that grant access
access_via_ticket	Cabin class required (e.g. Business) or N/A
has_showers	true / false / N/A
has_hot_food	true / false / N/A
has_bar	true / false / N/A
has_wifi	true / false / N/A
has_quiet_zone	true / false / N/A
hours	Human-readable or N/A

Column	Description
source_url	
extracted_at	

record_type: route_metadata

Column	Description
flight_number	
airline	
origin_iata	
destination_iata	
aircraft_type	
has_lie_flat_business	true / false / N/A
has_premium_economy	true / false / N/A
has_seatback_ife	true / false / N/A
has_wifi	true / false / N/A
meal_service	true / false / N/A
avg_duration_minutes	
frequency	Daily / Weekly / N/A
source_url	
extracted_at	

record_type: layover

Column	Description
airport_iata	
airport_name	

Column	Description
has_united_club	true / false / N/A
has_delta_sky_club	true / false / N/A
has_admirals_club	true / false / N/A
has_priority_pass_lounge	true / false / N/A
min_connection_minutes	Official MCT
recommended_connection_minutes	From scraped user reports or N/A
requires_terminal_transit	true / false / N/A
requires_immigration	true / false / N/A (relevant for intl connections)
source_url	
extracted_at	

record_type: program_rule

Column	Description
program	united / delta / american
rule_topic	award_expiry / upgrade_rules / stopover_policy / open_jaw / mixed_cabin
rule_summary	Plain text description (max 300 chars)
source_url	
extracted_at	

record_type: points_valuation

Column	Description
program	
cabin	economy / business / first
cents_per_point	e.g. 1.5

Column	Description
source	thepointsguy / onemileatatime
valuation_date	When the valuation was published
source_url	
extracted_at	

Crawl Engine Logic

Startup Sequence

python

```
1. Validate SCREENSHOTS_DIR and DATA_DIR are set (exit with clear error if blank)
2. Create both directories if they don't exist
3. Check Ollama is running: GET http://localhost:11434/api/tags
4. Check LLaVA model is available (if not: log warning, disable vision mode, continue)
5. Check for checkpoint.json → if found, resume from saved queue state
6. If no checkpoint: seed the queue with configured seed URLs
7. Fetch and parse sitemap.xml for each domain → add discovered URLs to queue
8. Start crawl loop
```

Per-Page Loop

python

```
while queue:
    url = queue.popleft()
    if url in visited: continue
    if not passes_scope_rules(url):
        log_skipped(url)
        continue

    visited.add(url)
    await random_delay(3, 8)          # polite delay

    page_data = await scrape_page(url) # dual-mode extraction
    save_to_sqlite(page_data)
    append_to_visit_log(page_data)

    new_links = extract_links(page_data)
    queue.extend(filter_links(new_links))

    save_checkpoint(queue, visited)   # overwrite checkpoint every page
```

Shutdown Sequence

python

1. Export SQLite → pandas → output.csv (fill N/A for all nulls)
2. Sweep and delete all remaining PNGs in SCREENSHOTS_DIR
3. Delete checkpoint.json (run completed cleanly)
4. Print summary: pages visited, pages failed, records extracted per type

Scope Rules

python

```
SCOPE = {
    "united.com":          {"max_depth": 5, "max_pages": 800},
    "delta.com":           {"max_depth": 4, "max_pages": 400},
    "aa.com":              {"max_depth": 4, "max_pages": 400},
    "ana.co.jp":           {"max_depth": 3, "max_pages": 80},
    "miles-and-more.com":  {"max_depth": 3, "max_pages": 80},
    "aircanada.com":       {"max_depth": 3, "max_pages": 80},
    "singaporeair.com":    {"max_depth": 3, "max_pages": 80},
    "turkishairlines.com": {"max_depth": 3, "max_pages": 80},
    "swiss.com":           {"max_depth": 3, "max_pages": 80},
    "creditcards.chase.com": {"max_depth": 3, "max_pages": 100},
    "americanexpress.com": {"max_depth": 3, "max_pages": 100},
    "citi.com":            {"max_depth": 3, "max_pages": 100},
    "capitalone.com":      {"max_depth": 3, "max_pages": 100},
    "biltrewards.com":     {"max_depth": 3, "max_pages": 100},
    "loungebuddy.com":     {"max_depth": 2, "max_pages": 150},
    "prioritypass.com":    {"max_depth": 2, "max_pages": 150},
    "google.com":          {"max_depth": 1, "max_pages": 50},
    "kayak.com":           {"max_depth": 1, "max_pages": 50},
    "thepointsguy.com":    {"max_depth": 2, "max_pages": 30},
    "onemileatatime.com":  {"max_depth": 2, "max_pages": 30},
}

URL_BLOCKLIST = [
    "/careers/", "/legal/", "/accessibility/", "/pressroom/",
    "/investor-relations/", "/sitemap", "robots.txt",
    "logout", "signin", "login", "account", "booking/confirm",
]
```

Scheduled Re-crawl (commented out by default)

```
python
```

```

# =====
# SCHEDULED RE-CRAWL – uncomment to enable
# Runs the full crawl on a schedule using APScheduler
# Requires: pip install apscheduler
# =====

# from apscheduler.schedulers.blocking import BlockingScheduler
#
# def run_scheduled_crawl():
#     print(f"[SCHEDULER] Starting scheduled crawl at {datetime.utcnow()}")
#     asyncio.run(main())
#
# scheduler = BlockingScheduler()
#
# # Run daily at 3:00 AM local time
# scheduler.add_job(run_scheduled_crawl, 'cron', hour=3, minute=0)
#
# # Alternative: run every 12 hours
# # scheduler.add_job(run_scheduled_crawl, 'interval', hours=12)
#
# # Alternative: run every Monday at 6:00 AM
# # scheduler.add_job(run_scheduled_crawl, 'cron', day_of_week='mon', hour=6)
#
# if __name__ == "__main__":
#     print("[SCHEDULER] Scheduled crawl active. Press Ctrl+C to stop.")
#     scheduler.start()
# =====

# For a one-time run (default), use:
if __name__ == "__main__":
    asyncio.run(main())

```

Logging

```

./data/
├─ visit_log.jsonl      ← one JSON record per page (all fields including success
                        flags)
├─ failed_urls.log      ← URLs where dom_success=False AND vision_success=False
├─ skipped_urls.log     ← URLs excluded by scope rules (with reason)
└─ scraper_YYYY-MM-DD.log ← rotating daily log, console-mirrored

```

Log format:

```
2025-05-28 14:32:01 [INFO] Visiting:
https://www.united.com/ual/en/us/fly/mileageplus/awards/...
2025-05-28 14:32:04 [INFO] DOM success. Extracted: award_flight (3 records)
2025-05-28 14:32:05 [WARN] [VISION_FAIL] https://... - Ollama timeout after 30s
2025-05-28 14:32:05 [INFO] Screenshot deleted: page_0042.png
2025-05-28 14:32:13 [ERROR] [BOTH_FAIL] https://... - added to failed_urls.log
```

Error Handling Rules (non-negotiable)

1. **No unhandled exceptions at the page level.** Every page visit is wrapped in a broad `try/except` that logs and continues.
 2. **Checkpoint is written after every page.** A crash mid-run loses at most one page of work.
 3. **Screenshot deletion is in a `finally` block.** Screenshots are always deleted even if LLaVA fails.
 4. **If Ollama is not running at startup**, vision mode is disabled and a clear warning is printed. DOM-only mode continues.
 5. **If a domain consistently blocks** (e.g. >10 consecutive failures), log a domain-level warning and move on to the next domain. Do not hammer it.
 6. **Rate limit headers** (`Retry-After`, 429 status): honor them. Back off and retry after the specified delay.
 7. **CSV export always runs**, even if some record types have zero rows. Empty types produce an empty section, not an error.
-

Output Notes for the Algorithm

- All numeric fields that could not be found are `N/A` as a string — parse defensively
- `extracted_at` is UTC ISO 8601 for all records — convert to local time as needed
- `dom_success` and `vision_success` columns are present on every row — use to weight confidence
- Records from `thepointsguy.com` / `onemileatatime.com` are reference valuations only — clearly marked in `source` column
- Transfer bonus data should be treated as time-sensitive — check `extracted_at` before using

- Cash prices from Google Flights / Kayak reflect the moment of scraping — these go stale fastest
-

Summary of What This Enables

The algorithm receiving this CSV can answer:

- Cheapest route A→B in pure cash across United, Delta, American
- Cheapest route A→B using award miles from any program
- Cheapest route A→B by converting Chase UR / Amex MR / Citi TY / CapOne / Bilt to airline miles (with current transfer ratios and any active bonuses)
- Best cents-per-mile value by program and cabin
- Which routing has lounge access at every layover, and which cards grant it
- Safest connection times by airport
- Whether a particular partner airline adds a fuel surcharge that kills the award value
- Whether an open-jaw or stopover routing is allowed and cheaper